

Prerequisite: Watch Network programming videos provided at Canvas.

Programming language: Java/Python are recommended but you may use any programming language which support client-server programming.

1 Introduction

In this exercise, using socket programming, you will develop a client-server application consisting of a Smart TV server and one or more remote-control clients. However, the development will contain multiple parts. At the end of each part, you will have "a bit better product".



Figure 1: Illustrative Smart TV scenario

2 Tasks

Part 1. Creating a Simple Smart TV solution (Week 37-39)

In this part the point is to "get started" and create a solution according to the specification using socket. You are allowed to be as sloppy as you want with the code structure. But you will need to fix it later. The task is to create a Smart-TV solution by designing the application-layer protocol, implementing the Smart TV as a TCP server, and a remote control as a TCP client.

1.1 Smart TV

1. The Smart TV is a server.
2. The TV has a fixed number of available channels C , where $C \geq 3$. You choose C in your implementation.
3. Channel 1 is the current channel when the server starts.
4. When the TV is turned OFF and then ON again, it remembers the previous channel. But restarting the server may reset the channel to 1.



5. While the TV is OFF, the only TV-control command that changes state is POWER ON.
6. While the TV is ON, the remote can read the channel count, read the current channel, set a channel, move one channel up and move one channel down.
7. Invalid channel and unknown commands must be handled.
8. Channel UP and DOWN use wrap-around: For example from channel with highest number, UP goes to 1; from 1, DOWN goes to channel with highest number.

1.2 Remote control

1. The remote is a TCP client.
2. A remote connects to one TV at a time.
3. It must allow the user to turn the TV ON/OFF, move the channel UP/DOWN.
4. Must support STATUS command. STATUS command should show whether the TV is ON or OFF.

1.3 Starting the application

When server process starts, it must take the TCP port as a command-line parameter. If no port is supplied, use port 1238. The client must take the server hostname/IP address and TCP port as command-line parameters. A command-line interface is sufficient. A GUI is optional.

1.4 Inspect the connection in Wireshark

Capture the session in Wireshark using filter.

1. What are the client and server IP addresses and TCP port numbers?
2. Where is the TCP three-way handshake?
3. Locate one Smart-TV application command in the TCP payload?
4. How does the TCP connection terminate in your capture?

Part 2. Refactoring and testing (Week 37-39)

An API divide responsibilities among modules and each module could contain many classes and methods. In this part your task is to improve the structure of the code according to good practices. As a result of that, the code will become easier to modify and test.

2.1 Refactor the Code

1. Think about the structure of the code. What can you improve?
2. Think about testing of SmartTV logic without network communication.
3. Sketch a **class diagram** (on a piece of paper or digitally if you wish). Use this as a thinking-board.

4. Extract the Smart TV logic into a separate class, so that it does not depend on communication in any way. The logic must have methods for the necessary TV functionality: `turnOn()`, `turnOff()`, `getNumberOfChannels()`, `getChannel()`, `setChannel(int channel)`.
5. The logic-class must not depend on any other classes.
6. It should be easy to change the messages in the protocol. For example, if you use "on" as a command for turning on the TV, it should be easy to change it to "turn-on" or something else. In this way you can make your protocol maintainable.
7. It should be easy to extend the system with new commands. In this way you can make your protocol extendable.
8. It should be easy to replace TCP with UDP. This is also an example of "making a protocol maintainable".

2.2 Write Unit Tests

If you have structured your code properly, you should be able to test the different parts separately. Write the following tests (at least 3):

1. Some logic-test, for example TV is OFF by default; Can't get channel list when the TV is OFF; Can turn on the TV; When turned on, getting the channel list works; Setting the channel to a valid channel works; Setting the channel to an invalid channel works;
2. Some message serialization tests, for example (adapt these to your protocol): "on" is interpreted as an "turn on" command ; "list" is interpreted as a "get channel list" command.; "Chuck is da best" is interpreted as an invalid command.
3. Some command interpretation tests, for example: Get-channel-list returns the expected value (number of channels); The TV changes its state from off to on when a turn-on command is executed and vice versa; the channel on the TV is switched when set-channel command is executed.

2.3 Server logs

Add simple console logging to show the user what the server is doing, for example:

- [STATE] Channel changed 3 -> 5

A logging framework is not required.

Part 3. Adding Multi-Client Support (Week 40-44)

Extend the application into a small distributed service. Several remote controls must be able to operate the same TV at the same time. Allow multiple simultaneous client-connections on the server: one TV supports multiple remotes connected at the same time. One idle or slow client must not stop other clients from using the TV. All clients control the same SmartTV object/state. Concurrent operations must not leave the TV in an invalid state. Use appropriate synchronization/thread-safe techniques. When one remote changes the channel, other connected remotes must receive the new channel. Clients must not poll the server all the time.

3 Cheating

The rules on cheating in exams also apply to compulsory activities that must be approved before the student can take the exam. Familiarize yourself with "What is cheating" and "Use of generative artificial intelligence" available at <https://i.ntnu.no/wiki/-/wiki/English/Cheating+on+exams>

4 Requirement for approval

1. Submit a Zip file of the exercise or link to the repository where the source code is stored. If you have some other way to share the solution, you can use that. Please make sure it must be accessible to the teacher.
2. Attend the checkpoint (meeting) with teacher/TA/SA and demonstrate your solution.

5 Deadline

Check Canvas.

6 Feedback: Optional

Your feedback will be appreciated.